Student Internship Programme (SIP)

CP3200 Project Report

at

Home Team Science and Technology Agency

Reporting period

05 2025 to 08 2025

by

Jordan Mitchell Chan

Department of Computer Science

School of Computing

National University of Singapore

2025/2026

Summary

HTX Q-team has developed a pipeline called Qmavis that can analyze videos and livestreams using an AI model hosted on Amazon Sagemaker to explain what is happening in the video or livestream. Additionally, Q-team has developed another, more refined platform called Genie that can do something similar for videos, texts and images. However, it's video analysis capabilities are less advanced the Qmavis's The end goal is to fully integrate Qmavis into Genie, modifying the various part of the Qmavis pipeline as needed to leverage the video analysis functionality of Qmavis

Subject Descriptors:

I.2.10: Computing Methodologies → Artificial Intelligence → Vision

I.4.8: Computing Methodologies → Image Processing and Computer Vision → Scene Analysis

H.3.3: Information Systems → Information Storage and Retrieval → Information Search and Retrieval

Keywords:

- Integration, Computer Vision

Acknowledgements

I would like to acknowledge HTX for providing me with this internship and a chance to grow in my abilities as a software engineer. I would also like to thank my mentors, Deryl and Zixing, for their extensive guidance and patience towards me while I was working on this project, despite their many responsibilities.

Table of Contents

Title

Summary

Acknowledgement

1. Introduction
   1.1 Qwen
   1.2 Genie
   1.3 Mavis
2. Learning stage
3. First Assignment
4. Second Assignment
   4.1 Making the Qmavis database cloud-based
   4.2 Modifying the audio transcription and report generation function
   4.3 Miscellaneous changes
5. Deploying the Docker image
   5.1 Shrinking the image
   5.2 Final deployment
6. Conclusion

Qwen

The Qwen models, developed by Alibaba Cloud and hosted on Hugging Face, are a family of large language models (LLMs) and multimodal models designed for advanced AI tasks. Qwen3, the latest series, includes dense and Mixture-of-Experts (MoE) models ranging from 0.6B to 235B parameters, with variants like Qwen3-235B-A22B and Qwen3-8B optimized for reasoning, instruction-following, and multilingual support across over 100 languages. These models excel in text generation, code generation, mathematics, and multimodal tasks (e.g., Qwen2.5-Omni and Qwen VLo handle text, audio, images, and videos). They support features like thinking and non-thinking modes for flexible problem-solving, with open-source models licensed under Apache 2.0. Qwen models are accessible via Hugging Face Transformers, ModelScope, and tools like vLLM, with APIs available through Alibaba Cloud Model Studio

Genie

Within HTX, Q-team is the department responsible for prototyping various ideas that may be selected for further development. Among these projects, one. Of them is called Genie. Genie is a multi-modal platform that can take in various types of information such as images, videos and text documents and uses a Qwen AI model to analyze it and generate a case study. This is useful in situation such as a law enforcement case, where the police can feed in the various types of data, such as surveillance and body cam footage and get a comprehensive summary of events. However, the genie platform has certain limitations, inherent to the Qwen model being used. Namely, there is a token limit for videos per query. This means that videos exceeding a certain length (and resolution) will not be able to be processed in a single query, making Genie quite limited in terms of real-world usage.

Mavis

Qmavis is another video processing pipeline developed by Q-team. Qmavis can take in either a single video, several videos (and use Qwen to analyze them in the context of each other) or a livestream and uses Qwen to summarize what is happening, transcribe the audio and generate a report if necessary. Qmavis overcomes the token limitation by splitting a video

into more manageable sized chunks(length selected by user) and feeding them in one by one. In its original design, the user will upload a video from their local machine and the video will still be stored locally while being processed by Qmavis.

Learning stage

At the start of the internship, I spent a week or so familiarizing myself with the tools I would need, mainly Docker, Docker compose and FastAPI. Docker is an open-source platform that uses containerization to package applications and their dependencies into lightweight, portable containers. Containers run consistently across different environments (e.g., development, testing, production) by isolating the application's code, libraries, and runtime into an "image". This image can then be used to spin up a container for the application, ensuring it behaves the same regardless of the underlying system. This simplifies deployment and reduces "it works on my machine" issues.

Docker Compose is a tool for defining and managing multi-container Docker applications. It uses a YAML file to configure services, networks, and volumes, allowing you to spin up multiple containers (e.g., app server, database, cache) with a single command. It simplifies orchestrating complex applications by specifying how containers interact. Learning the nuances of Docker images and containers was somewhat tricky and required a bit of time to fully understand as I was completely new to it.

Meanwhile, FastAPI is a modern, high-performance web framework for building APIs with Python, based on standard Python type hints. It is designed for speed, leveraging asynchronous programming (using asyncio) and libraries like Starlette and Pydantic to deliver performance comparable to Node.js or Go. FastAPI automatically generates interactive API documentation (via Swagger UI or ReDoc) and validates data with Pydantic models, ensuring type safety and reducing errors.

First Assignment

In the first part of my internship, for about 2 weeks, I was tasked with working on the Genie project. My job was to figure out ways to circumvent the token restriction that limited the size and quality of videos that could be uploaded. I was working with a partner and we

needed a precise figure on how long a video could be when uploaded and what quality. This required trial-and-error testing to determine. I also had to familiarize myself with the Genie code in order to implement these testing strategies.

Second Assignment

I was subsequently pulled off the Genie project and assigned to the Qmavis project. The idea behind Qmavis was that there was an existing project designed specifically to handle video input, with a methodology already in place to deal with issues such as length and tokenization limits. The end goal is to see if Qmavis can be integrated seamlessly into the Genie platform so that Genie can leverage Qmavis's capabilities. My job was to see how that could be done and address all compatibility issues along the way. The first key milestone was to make everything cloud-based.

Section 4.1

Genie is already a deployed project and leverages cloud-based technology. Uploaded files were stored in an Amazon S3 bucket and all processing was done by the Qwen mode. Qmavis originally was a prototype and there was no integration at all with the Amazon S3 service. All uploaded files would be stored locally, which took up a lot of space. After taking a day or so to understand the code base and figure out what everything did, I rewrote many of the functions to access the HTX's S3 bucket using my supervisor's key and store all uploaded files there. Subsequently, I also rewrote all the processing functions to call the files from the S3 bucket instead of looking for them locally. To do this, I used my knowledge of OOP to create a new class that would handle all file access and storage in the S3 bucket, meanwhile all other classes and methods would call methods from a global instance of this new class. While doing this, I faced several challenges. The main one was debugging. Because there was a high degree of interdependence between utility function and the general complexity of the project in general, a lot of bugs kept cropping up while I was rewriting the functions. Not only that, but I had to troubleshoot some issues from the previous intern(as Qmavis had been partially worked on by a previous intern and when I pulled a copy of the project from his branch, there were several issues). For example, there was a typo in the docker image tag, which was very confusing as composing the image simply kept throwing the error version not

found and I was helpless until I found my supervisor to find out which was the correct version

Section 4.2

The next step was to modify the audio transcription and report generation feature from using a local library to calling the Whisper and Qwen models respectively hosted on Amazon Sagemaker to extract the audio from videos. This was achieved using another new class that would handle interactions between the processed video file(that have undergone chunking) and the Qwen model. The tricky part here was that since the python libraries still need a local copy of the video to operate on, I still needed to download a temporary local copy from the S3 bucket, process them, then submit it to the Qwen model on Sagemaker before deleting the temporary copy. The drawback to doing this was that Qmavis's ability to process livestreams was removed, however my supervisor decided to prioritise the integration of Qmavis into Genie at the cost of the livestream processing feature. There were also many bugs that kept cropping up during this process that had to be fixed, similar to the above section.

Section 4.3

I also made some miscellaneous changes. For example, previously there was no cleanup feature so files would endlessly accumulate, both in the local machine and in the S3 bucket after I made the initial changes. To address this, I implemented a cleanup feature such that a limit can be set for the number of files in the S3 bucket. Once a file is uploaded such that the limit is exceeded, the oldest upload will be deleted to make space for the new file. I also added some increased parallelization of different functions so that in the future the pipeline has more potential for increased efficiency.

Deploying the Docker image

After finishing the above, there was a need to deploy the Docker image. However, the image size was unusually large, around 30GB whereas my supervisor indicated it should be no bigger than 1GB or 2GB. He tasked me to figure out what was wrong and how to shrink it.

Section 5.1

The 30GB image was the latest one and built on top of the original Docker image for the project. However, as I had learnt, docker images are immutable and changes made simply superimpose a new layer over old ones(Docker images are composed of layers which are responsible for different things like installing dependencies, environment, etc). Thus the first thing I did was build a new Docker image for the project via a new Dockerfile instead of pulling the old image. This was difficult as I was still quite new to Docker(up till now I just used the same few commands) and many small things were wrong/missing that prevented the application from running properly and were not straightforward to debug.

After that, the image size was still almost 10GB, still well in excess of the target. To fix this, I used trial and error to remove as many redundant dependencies as I could. There were many, previously needed when Qmavis used local libraries instead of the Qwen model on Sagemaker to process videos, which I was able to find and remove. Although relatively straightforward, it was still not completely trivial as there were many libraries that I had never seen before and I had to work out what each of them did.

Section 5.2

The final step was to push all the changes to the GitHub repo, and deploy the container onto Genexis, the HTX platform where all their software projects are deployed.

Conclusion

Overall, this was a rewarding internship where I learn a great deal about the software engineering process, specifically the process of debugging, new technologies such as Docker and softer skills such as how to be independent in a real life project.

Declaration: AI was used in certain parts of this report to assist with explaining existing technologies such as Docker.